

# Axon Flow

Hierarchical Mind-Mapping Platform

## Software Requirements Specification

Version 1.0 | Pre-Development Reference

**Prepared By:** Akshat Raj (23scse1010833)  
Pavan Soni (23scse1012576)  
**Department:** Computer Science & Engineering  
**Institution:** Galgotias University  
**Status:** In Development  
**Date:** May 26, 2026

### Tech Stack

**Frontend:** React 18 + React Flow (@xyflow/react) +  
D3-Hierarchy  
**Backend:** Node.js + Express.js (REST API)  
**Database:** MongoDB Atlas (Mongoose ODM)  
**Auth:** Clerk (JWT / OAuth)  
**Layout Engine:** D3-Hierarchy (stratify + tree)

## 0 Contents

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                               | <b>4</b>  |
| 1.1      | Purpose . . . . .                                 | 4         |
| 1.2      | Project Overview . . . . .                        | 4         |
| 1.3      | Scope . . . . .                                   | 4         |
| 1.4      | Definitions and Abbreviations . . . . .           | 4         |
| <b>2</b> | <b>Overall System Description</b>                 | <b>5</b>  |
| 2.1      | System Perspective . . . . .                      | 5         |
| 2.2      | Application Route Structure . . . . .             | 5         |
| 2.3      | User Classes . . . . .                            | 5         |
| 2.4      | Operating Environment . . . . .                   | 5         |
| <b>3</b> | <b>Problem Statement</b>                          | <b>6</b>  |
| 3.1      | Existing System Limitations . . . . .             | 6         |
| 3.2      | How Axon Flow Solves These . . . . .              | 6         |
| <b>4</b> | <b>Functional Requirements</b>                    | <b>6</b>  |
| 4.1      | FR-1: Authentication & Route Protection . . . . . | 7         |
| 4.2      | FR-2: Workspace & Folder Management . . . . .     | 7         |
| 4.3      | FR-3: Map Lifecycle . . . . .                     | 7         |
| 4.4      | FR-4: Canvas & Node Interaction . . . . .         | 7         |
| 4.5      | FR-5: Bulk Import . . . . .                       | 8         |
| 4.6      | FR-6: Node Enrichment . . . . .                   | 8         |
| <b>5</b> | <b>High-Level Design (HLD)</b>                    | <b>8</b>  |
| 5.1      | System Architecture . . . . .                     | 9         |
| 5.2      | Component Map . . . . .                           | 9         |
| 5.3      | Data Flow Diagrams (DFD) . . . . .                | 10        |
| 5.3.1    | Level 0 DFD (Context Diagram) . . . . .           | 10        |
| 5.3.2    | Level 1 DFD . . . . .                             | 11        |
| <b>6</b> | <b>Detailed Data Flow — Flat DB to Canvas</b>     | <b>11</b> |
| 6.1      | The Core Challenge . . . . .                      | 11        |
| 6.2      | Step-by-Step: Opening a Map . . . . .             | 11        |
| 6.2.1    | Concrete Example . . . . .                        | 12        |
| 6.3      | Step-by-Step: Dragging a Node . . . . .           | 13        |
| <b>7</b> | <b>Low-Level Design (LLD)</b>                     | <b>14</b> |
| 7.1      | Database Schema . . . . .                         | 16        |
| 7.1.1    | Collection: <b>nodes</b> . . . . .                | 16        |

|           |                                          |           |
|-----------|------------------------------------------|-----------|
| 7.1.2     | Collection: <b>maps</b>                  | 16        |
| 7.1.3     | Collection: <b>folders</b>               | 16        |
| 7.2       | ER Diagram                               | 17        |
| 7.3       | API Endpoints — Full Reference           | 17        |
| 7.3.1     | Map Routes (/api/maps)                   | 17        |
| 7.3.2     | Node Routes (/api/nodes)                 | 17        |
| 7.3.3     | Folder Routes (/api/folders)             | 18        |
| 7.4       | D3 Layout Algorithm (LLD Detail)         | 18        |
| 7.5       | Bulk Import Parser Algorithm             | 19        |
| <b>8</b>  | <b>Frontend &amp; Interaction Design</b> | <b>19</b> |
| 8.1       | Hook Architecture                        | 19        |
| 8.2       | useCanvasState: The Sync Engine          | 20        |
| 8.3       | D3StyleNode: The Node Visual Component   | 20        |
| 8.4       | Keyboard Interaction Map                 | 20        |
| 8.5       | Context Menu — FloatingToolbar           | 21        |
| <b>9</b>  | <b>Non-Functional Requirements</b>       | <b>21</b> |
| <b>10</b> | <b>Testing Requirements</b>              | <b>22</b> |
| 10.1      | Strategy                                 | 22        |
| 10.2      | Key Test Cases                           | 22        |
| <b>11</b> | <b>Deployment &amp; Planning</b>         | <b>22</b> |
| 11.1      | Deployment Architecture                  | 23        |
| 11.2      | Development Milestones                   | 23        |
| 11.3      | Future Enhancements                      | 23        |
| <b>12</b> | <b>Conclusion</b>                        | <b>23</b> |
| <b>13</b> | <b>Project Outputs &amp; Screenshots</b> | <b>25</b> |

# 1 Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) documents the complete functional, architectural, and interaction requirements for **Axon Flow**, a web-based hierarchical mind-mapping application. It serves as the single authoritative reference for design, development, and testing decisions.

## 1.2 Project Overview

Axon Flow is a full-stack, browser-native mind-mapping tool. Users create visual, hierarchical maps of ideas, notes, and plans on an infinite, pannable canvas. The system uses **React Flow** (`@xyflow/react`) as the canvas layer and a custom **D3-Hierarchy** layout engine to automatically position nodes in three modes: Horizontal (default), Vertical, and Radial. Every node is stored as an *independent flat document* in MongoDB, referenced via `parentId`, and reconstructed into a visual tree by the frontend at runtime.

## 1.3 Scope

- Interactive drag-and-drop mind-map canvas with three layout modes.
- Workspace → Folder → Map organizational hierarchy.
- Bulk node import by pasting indented plain text.
- Node enrichment: notes, hyperlinks, file attachments, and status badges.
- Secure per-user data isolation via Clerk authentication.
- Dashboard, My Maps, Workspace, Favorites, Trash, and Editor views.

## 1.4 Definitions and Abbreviations

| Term                 | Definition                                                                           |
|----------------------|--------------------------------------------------------------------------------------|
| <b>SRS</b>           | Software Requirements Specification                                                  |
| <b>HLD</b>           | High-Level Design                                                                    |
| <b>LLD</b>           | Low-Level Design                                                                     |
| <b>React Flow</b>    | The <code>@xyflow/react</code> library used for the interactive canvas               |
| <b>Backend Node</b>  | A flat MongoDB document representing one mind-map bubble                             |
| <b>D3-Stratify</b>   | Converts a flat array with <code>parentId</code> references into a D3 tree hierarchy |
| <b>Drag-Reparent</b> | Dropping one node onto another to re-assign its parent                               |

---

**ODM**                      Object Data Mapper (Mongoose)

---

## 2 Overall System Description

### 2.1 System Perspective

Axon Flow is a standalone client-server application. The React frontend communicates with the Node.js/Express backend over REST. The backend connects to MongoDB Atlas for data persistence and offloads authentication to Clerk. The system is completely stateless at the HTTP layer; all session context is derived from JWT tokens.

### 2.2 Application Route Structure

All routes are guarded by the `ProtectedRoute` component in `App.jsx`. Unauthenticated users are redirected to `/welcome`.

| URL Path                       | Component                              |
|--------------------------------|----------------------------------------|
| <code>/welcome</code>          | <code>LandingPage</code>               |
| <code>/</code>                 | <code>Dashboard</code>                 |
| <code>/maps</code>             | <code>MyMaps</code>                    |
| <code>/workspaces</code>       | <code>WorkspaceView</code>             |
| <code>/workspaces/:name</code> | <code>WorkspaceView</code> (filtered)  |
| <code>/favorites</code>        | <code>MyMaps</code> (filter=favorites) |
| <code>/trash</code>            | <code>MyMaps</code> (filter=trash)     |
| <code>/map/:id</code>          | <code>Editor</code> (Canvas)           |

### 2.3 User Classes

| User Class           | Goals                                                                                 |
|----------------------|---------------------------------------------------------------------------------------|
| <b>Students</b>      | Organize study material, import notes via Bulk Import, visualize topic relationships. |
| <b>Professionals</b> | Map project workflows, tag and organise maps into workspaces and folders.             |
| <b>Researchers</b>   | Build deep concept trees with notes, links, and file attachments per node.            |

### 2.4 Operating Environment

- **Client:** Chrome, Firefox, or Edge (ES2020+, JavaScript enabled, 1280×720 minimum).

- **Server:** Node.js v18+, Express.js on any Linux-based host (Render, Railway, AWS EC2).
- **Database:** MongoDB Atlas (Shared or Serverless tier).
- **Auth Provider:** Clerk (JWT-based, free tier).
- **Frontend Host:** Vercel (optimized for Vite + React).

## 3 Problem Statement

### 3.1 Existing System Limitations

Existing web-based mind-mapping tools share common architectural weaknesses:

- **Nested-Document Storage:** Storing the entire map tree as one deeply nested JSON document makes partial updates expensive (must rewrite the whole document).
- **DOM Re-render Cost:** Tools that render maps directly as nested HTML `divs` suffer heavy re-paints when any node is dragged.
- **Manual Data Entry:** No mechanism to import a large outline quickly.
- **No Rich Node Context:** Most tools only allow text; no inline notes, links, or files.

### 3.2 How Axon Flow Solves These

- **Flat-Tree Storage:** Each node is an independent MongoDB document. Updating one node issues a single `PATCH /api/nodes/:id` call.
- **React Flow + Virtual DOM:** Only the modified node subtree triggers a React reconciliation cycle. `useMemo` and `React.memo` gate unnecessary re-renders.
- **D3-Hierarchy Layout:** All spatial positions are computed by a battle-tested layout engine, not hand-coded.
- **Bulk Import Parser:** A single paste of indented text creates an entire subtree in one `/api/nodes/bulk-create` request.
- **Rich Node Data:** Each node stores `notes[]`, `links[]`, `files[]`, and a `status` badge.

## 4 Functional Requirements

## 4.1 FR-1: Authentication & Route Protection

### Authentication Module

- FR-1.1** The system shall redirect unauthenticated users from any protected route to `/welcome`.
- FR-1.2** The backend shall extract `userId` from every request via the Clerk JWT or the development `x-user-id` header.
- FR-1.3** Every API endpoint shall return HTTP 401 if `req.auth.userId` is absent.

## 4.2 FR-2: Workspace & Folder Management

- FR-2.1** The system shall support a three-tier organisational hierarchy: **Workspace** (string label on Map/Folder) → **Folder** (MongoDB document with optional `parentId`) → **Map**.
- FR-2.2** Workspaces shall be derivable by querying distinct `workspace` values owned by the user.
- FR-2.3** Renaming a workspace shall cascade to all maps and folders sharing that workspace string.
- FR-2.4** Deleting a folder shall recursively delete all child folders and their maps.

## 4.3 FR-3: Map Lifecycle

- FR-3.1** Creating a map shall atomically create a Map document and its root Node document.
- FR-3.2** Maps shall support: **Favourite** (`isFavorite`), **Trash** (`isTrashed`), and custom **Tags**.
- FR-3.3** Duplicating a map shall deep-clone all node documents with new IDs while preserving the hierarchy structure.
- FR-3.4** Bulk attribute update shall allow trashing or tagging multiple maps in one API call.

## 4.4 FR-4: Canvas & Node Interaction

- FR-4.1** The canvas shall support three layout modes: **Horizontal** (default column-width layout), **Vertical** (top-down tree), **Radial** (polar coordinates).
- FR-4.2** Layout shall be computed by `buildReactFlowData()` in `flowUtils.js` using `d3-hierarchy`.
- FR-4.3** Keyboard shortcuts on a focused node: **Tab** adds a child, **Enter** adds a sibling, **F2** renames, **Delete** removes.
- FR-4.4** Nodes shall support collapsing their subtree via the coloured circle indicator (`isExpanded` field).

- FR-4.5** Dragging one node onto another shall **reparent** it (update `parentId`). The system must prevent cycles (dragging onto a descendant).
- FR-4.6** Dragging a node to empty space shall save its new `x`, `y` coordinates to the database.
- FR-4.7** Four colour palettes shall be available: **Mono**, **Vibrant**, **Pastel**, **Neon**.

## 4.5 FR-5: Bulk Import

- FR-5.1** The `parseIndentedText()` function shall parse tab- or space-indented text into a flat array of `{id, parentId, name}` objects using an indentation-tracking stack.
- FR-5.2** The parsed array shall be sent to `POST /api/nodes/bulk-create` in a single HTTP request.
- FR-5.3** The system shall also support pasting plain-text directly on the canvas (global `paste` event), which triggers the same parser.

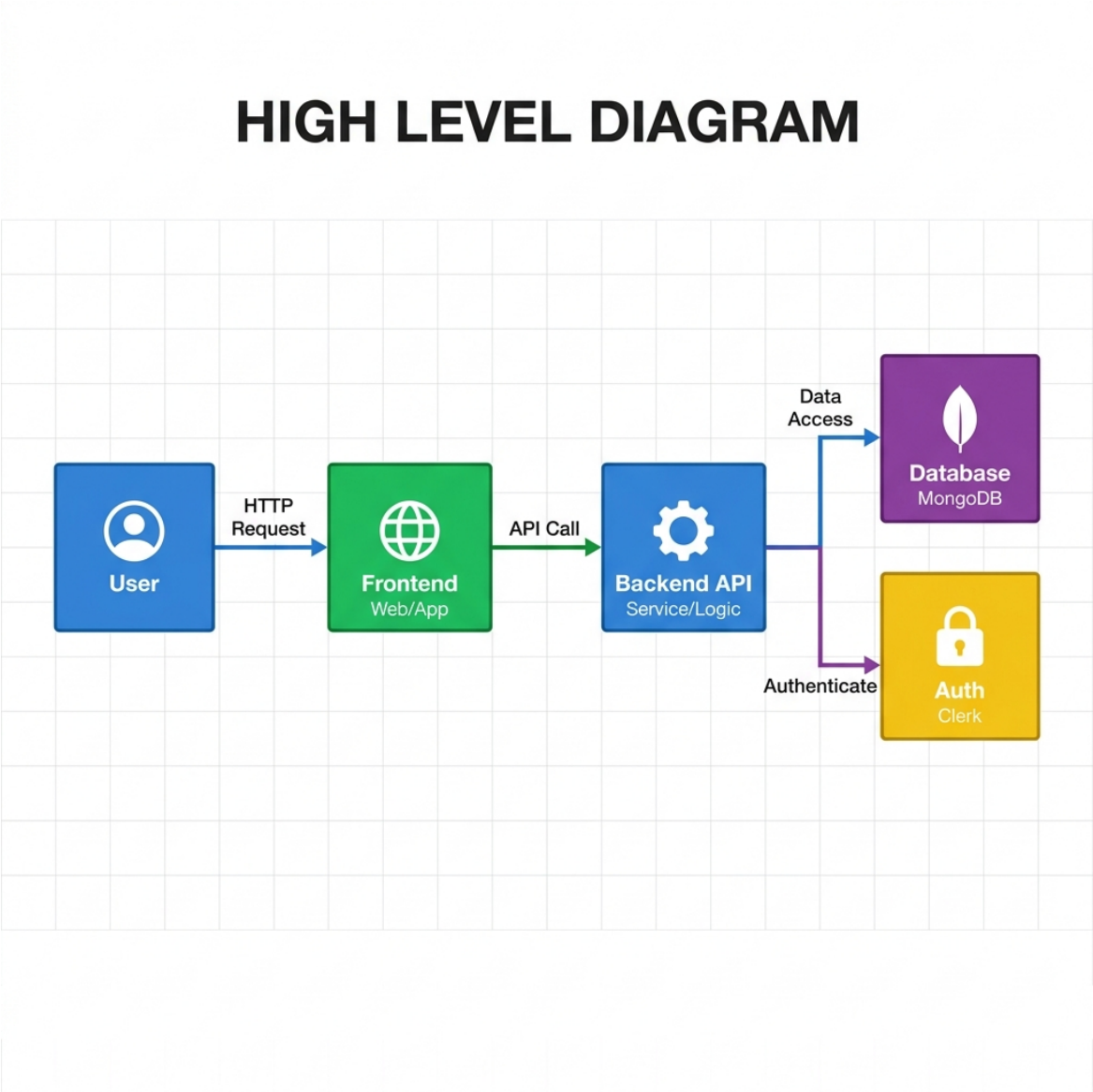
## 4.6 FR-6: Node Enrichment

- FR-6.1** Each node shall carry: `notes[]`, `links[{title, url}]`, `files[{fileName, fileUrl}]`, and a `status` string.
- FR-6.2** Valid status values and their icons: **reading** ([Icon]), **completed** ([Icon]), **incomplete** ([Icon]), **important** ([Icon]), **revise** ([Icon]).
- FR-6.3** A node with notes shall display a [Icon] suffix in its label.

# 5 High-Level Design (HLD)



5.1 System Architecture



5.2 Component Map

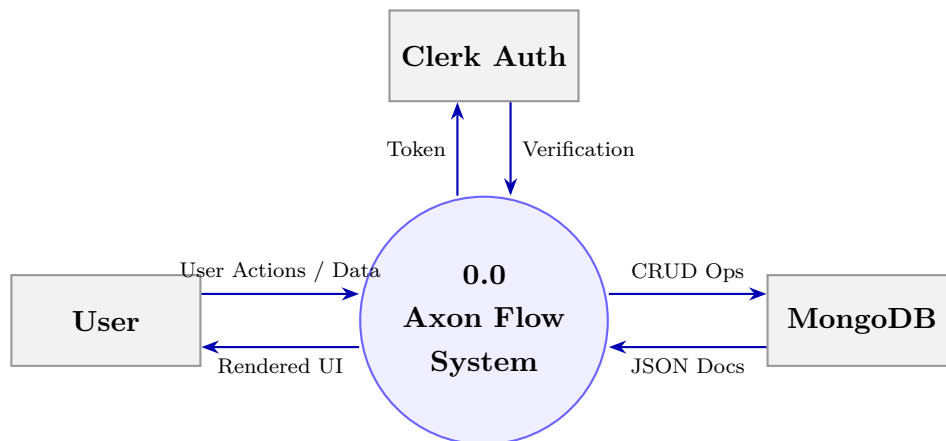
| Layer          | File / Module       | Responsibility                                     |
|----------------|---------------------|----------------------------------------------------|
| Frontend Pages | Dashboard.jsx       | Top-level page views, route endpoints              |
|                | Editor.jsx          |                                                    |
|                | MyMaps.jsx          |                                                    |
|                | WorkspaceView.jsx   |                                                    |
| Canvas Core    | CanvasContainer.jsx | Central hub; wires all hooks and renders ReactFlow |

|                      |                                                                                                        |                                                               |
|----------------------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| <b>Canvas</b>        | <code>useCanvasState.js</code>                                                                         | State sync, event handling, CRUD actions, drag                |
| <b>Hooks</b>         | <code>useCanvasEvents.js</code><br><code>useCanvasActions.js</code><br><code>useDragReparent.js</code> | <code>reparent</code>                                         |
| <b>Layout Engine</b> | <code>flowUtils.js</code>                                                                              | D3 stratify + tree layout; 3 modes; colour palette assignment |
| <b>Node Visual</b>   | <code>D3StyleNode.jsx</code>                                                                           | Renders one node bubble with inline edit, status icons, glow  |
| <b>Edge Visual</b>   | <code>D3BezierEdge.jsx</code>                                                                          | Animated bezier connector between nodes                       |
| <b>Backend Entry</b> | <code>app.js</code><br><code>index.js</code>                                                           | / Express middleware, CORS, auth mock, route mount            |
| <b>Routes</b>        | <code>mapRoutes.js</code><br><code>nodeRoutes.js</code><br><code>folderRoutes.js</code>                | URL-to-controller mapping                                     |
| <b>Controllers</b>   | <code>mapController.js</code><br><code>nodeController.js</code><br><code>folderController.js</code>    | HTTP req/res; delegates to services                           |
| <b>Database</b>      | <code>Map.js</code>                                                                                    | Mongoose schemas                                              |
| <b>Models</b>        | <code>Node.js</code><br><code>Folder.js</code>                                                         |                                                               |

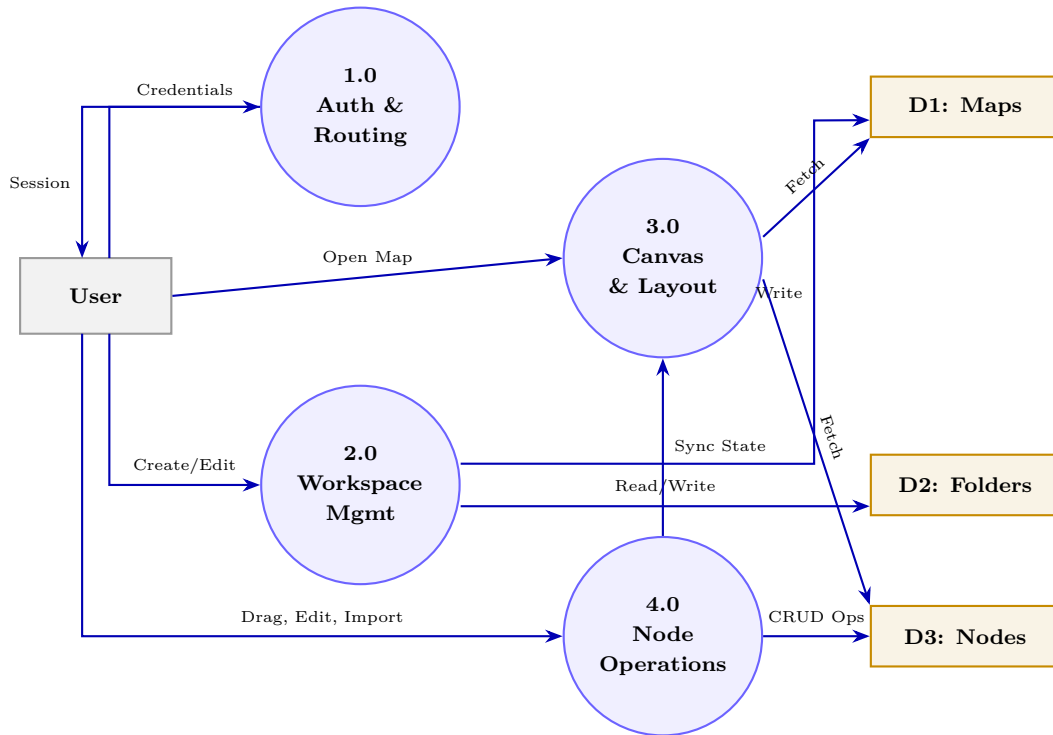
---

## 5.3 Data Flow Diagrams (DFD)

### 5.3.1 Level 0 DFD (Context Diagram)



### 5.3.2 Level 1 DFD

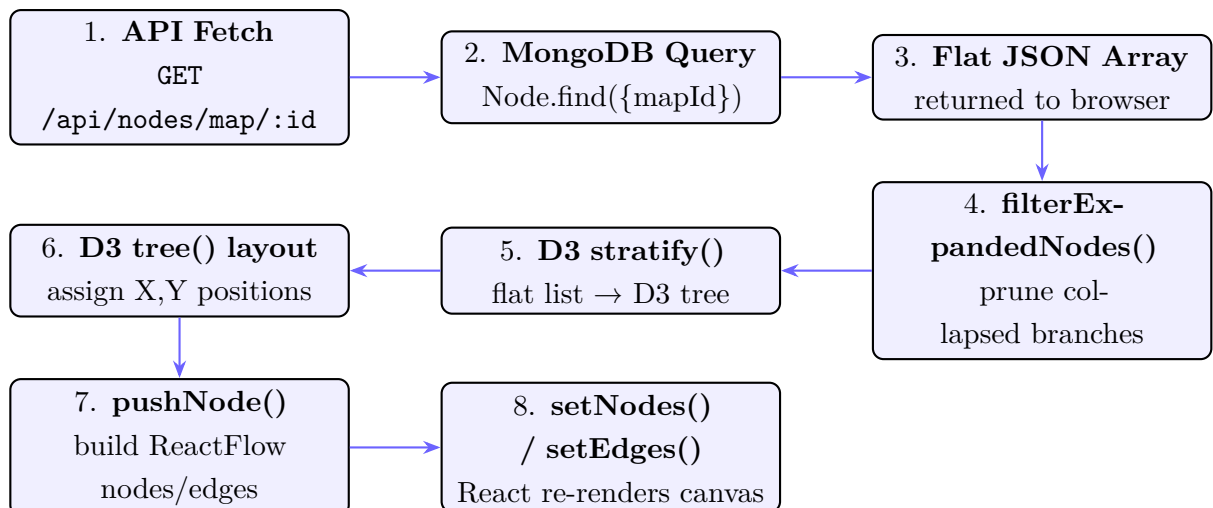


## 6 Detailed Data Flow — Flat DB to Canvas

### 6.1 The Core Challenge

A mind map is a tree. Trees stored as nested documents create expensive writes (the whole document must be replaced for one change). Axon Flow stores nodes as a **flat list**; each node knows only its `parentId`. The frontend reconstructs the tree at runtime.

### 6.2 Step-by-Step: Opening a Map



### 6.2.1 Concrete Example

MongoDB stores these three documents (flat):

Listing 1: Flat nodes stored in MongoDB

```

1 [
2   { "_id": "NODE_101", "mapId": "MAP_001", "parentId": null,
3     "name": "AI Research", "x": 0, "y": 0 },
4   { "_id": "NODE_102", "mapId": "MAP_001", "parentId": "
    NODE_101", "name": "Deep Learning", "x": 300, "y": -60 },
5   { "_id": "NODE_103", "mapId": "MAP_001", "parentId": "
    NODE_101", "name": "NLP", "x": 300, "y": 60 }
6 ]

```

Frontend calls `buildReactFlowData()` which runs D3 stratify:

Listing 2: D3 stratify in flowUtils.js

```

1 const root = stratify()
2   .id(d => d.id)
3   .parentId(d => d.parentId)(backendNodes); // flat -> D3
4     hierarchy
5
6 // D3 tree() assigns .x and .y to each node in the hierarchy.
7 // In Horizontal mode, colX Map controls the X-axis per depth
8 // level.
9 // measureTextPx() ensures wide node names get wider columns.
10 const layout = tree().nodeSize([1, 1])
11   .separation((a, b) => (hA + hB) / 2 + 40); // dynamic gap
12 layout(root);

```

Result sent to React Flow:

Listing 3: ReactFlow node objects after layout

```

1 [
2   { "id": "NODE_101", "type": "d3Node", "position": {"x": 0,
3     "y": 0}, "data": {"name": "AI Research", "depth": 0} },
4   { "id": "NODE_102", "type": "d3Node", "position": {"x": 380,
5     "y": -45}, "data": {"name": "Deep Learning", "depth": 1} },
6   { "id": "NODE_103", "type": "d3Node", "position": {"x": 380,
7     "y": 45}, "data": {"name": "NLP", "depth": 1} }
8 ]

```

**Edge Generation & React Flow Rendering:** Once the nodes have their absolute

positions from D3, the frontend generates React Flow `nodes` and `edges` arrays. Every child node automatically creates an edge connecting it back to its parent.

Listing 4: Generating Nodes and Edges for React Flow

```
1  const flowNodes = [];  
2  const flowEdges = [];  
3  
4  // 'root' represents the D3 hierarchy tree  
5  root.each(node => {  
6    // 1. Create the React Flow Node  
7    flowNodes.push({  
8      id: node.id,  
9      type: 'd3Node', // Maps to D3StyleNode.jsx  
10     position: { x: node.x, y: node.y },  
11     data: { name: node.data.name, depth: node.depth }  
12   });  
13  
14   // 2. Create the Edge connecting to its parent  
15   if (node.parent) {  
16     flowEdges.push({  
17       id: `edge-${node.parent.id}-${node.id}`,  
18       source: node.parent.id, // Parent Node ID  
19       target: node.id,        // Child Node ID  
20       type: 'd3Edge',         // Maps to D3BezierEdge.jsx  
21       animated: true,  
22       style: { stroke: '#6C63FF', strokeWidth: 2 }  
23     });  
24   }  
25 });  
26  
27 // React Flow consumes these arrays to render the canvas  
28 setNodes(flowNodes);  
29 setEdges(flowEdges);
```

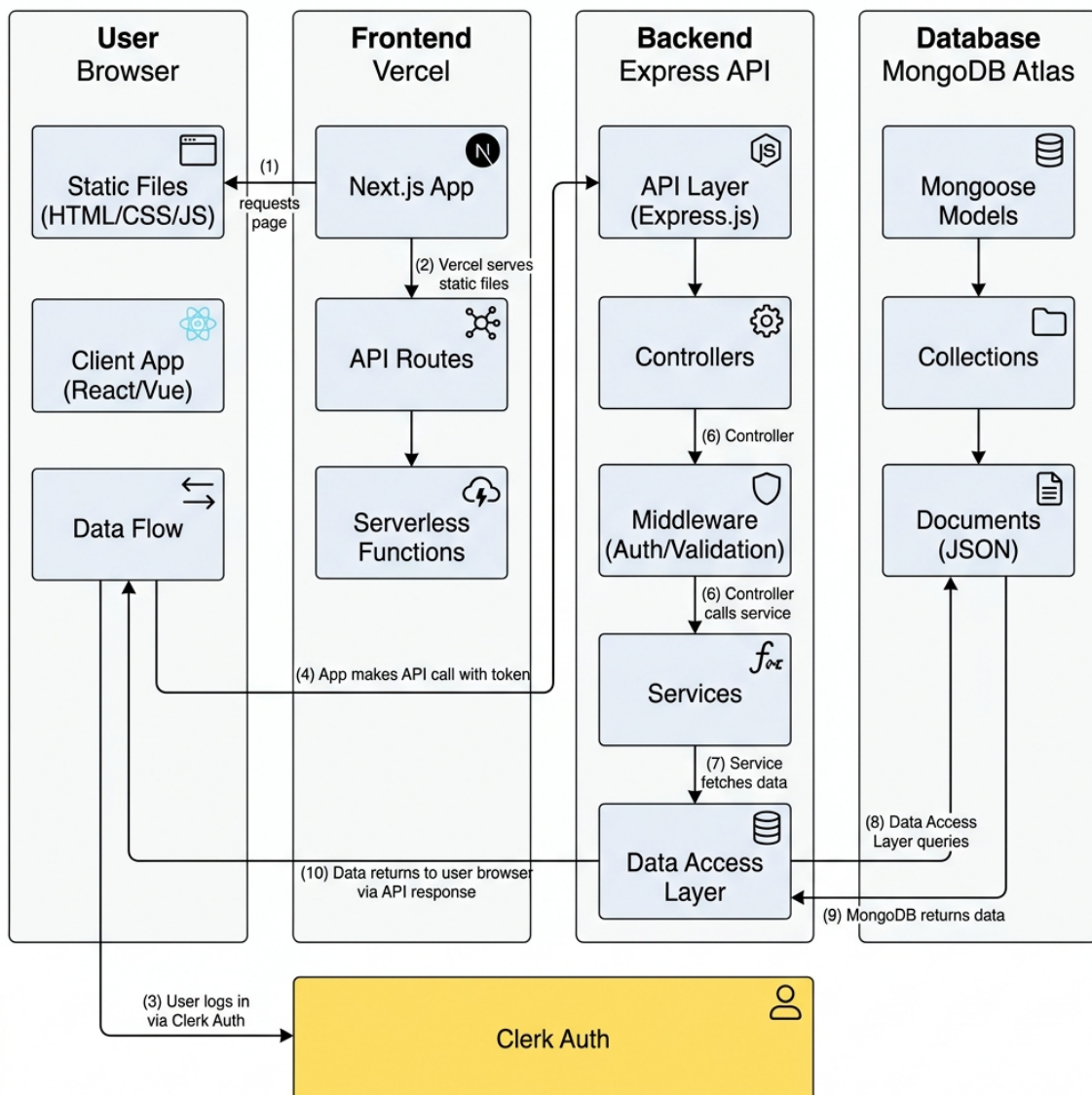
## 6.3 Step-by-Step: Dragging a Node

1. `onNodeDrag` fires in `useDragReparent.js`. A hit-test checks if the dragged node's centre overlaps any other node bounding box (+24px padding).
2. If a valid drop target is found (not a descendant, not current parent), it is highlighted with a blue pulsing ring (`isDropTarget`).
3. `onNodeDragStop` fires. If a drop target exists:
  - **Optimistic update:** `setBackendNodes()` immediately updates `parentId`

in local state → canvas re-renders instantly.

- **Persist:** PATCH /api/nodes/:id {parentId: target} sent to backend.
  - **Rollback:** If the API call fails, the frontend re-fetches the full node list to revert.
4. If *no* drop target: PATCH /api/nodes/:id/position {x, y} saves the new free-position.

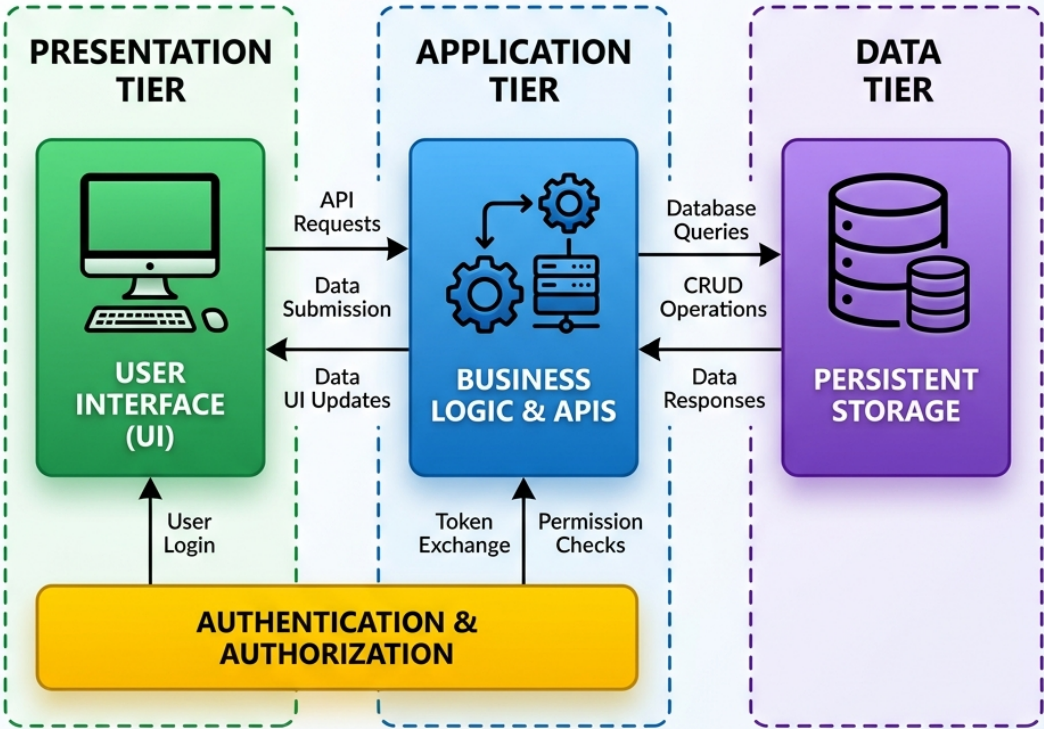
## 7 Low-Level Design (LLD)



Three-Tier Architecture Diagram



# THREE-TIER ARCHITECTURE DIAGRAM



## 7.1 Database Schema

### 7.1.1 Collection: nodes

| Field             | Type                  | Description                                                   |
|-------------------|-----------------------|---------------------------------------------------------------|
| <b>mapId</b>      | ObjectId              | FK → <b>maps</b> collection. Scopes node to a single map.     |
| <b>userId</b>     | String                | Clerk user ID. Enforces per-user data isolation.              |
| <b>parentId</b>   | ObjectId null         | FK → <b>nodes</b> (self-ref). <b>null</b> = root node.        |
| <b>name</b>       | String                | Text content of the node bubble.                              |
| <b>isExpanded</b> | Boolean               | Whether children are visible on canvas.                       |
| <b>notes</b>      | [String]              | Deep-dive notes attached to this node.                        |
| <b>links</b>      | [{title, url}]        | Hyperlinks attached to this node.                             |
| <b>files</b>      | [{fileName, fileUrl}] | File attachments (stored via <code>/api/files</code> upload). |
| <b>status</b>     | String null           | Badge: reading, completed, incomplete, important, revise.     |
| <b>x, y</b>       | Number                | Canvas position coordinates (for free-drag placement).        |

### 7.1.2 Collection: maps

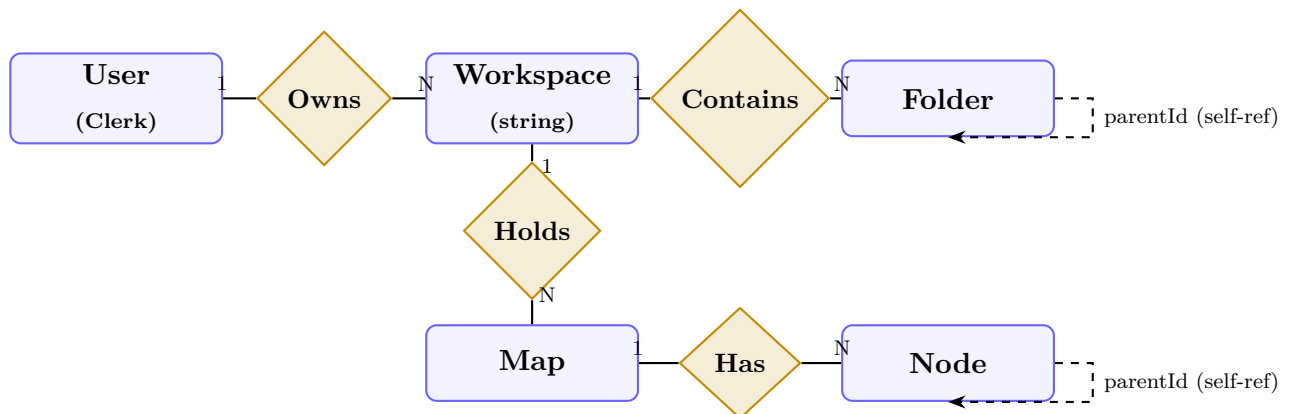
| Field                 | Type          | Description                                           |
|-----------------------|---------------|-------------------------------------------------------|
| <b>name</b>           | String        | Display name of the map.                              |
| <b>userId</b>         | String        | Owner's Clerk ID.                                     |
| <b>rootNodeId</b>     | ObjectId      | Reference to the map's root node.                     |
| <b>workspace</b>      | String null   | Top-level organisational label (e.g., "Research").    |
| <b>folderId</b>       | ObjectId null | Optional FK → <b>folders</b> for nested organisation. |
| <b>isFavorite</b>     | Boolean       | Pinned to Favorites view.                             |
| <b>isTrashed</b>      | Boolean       | Soft-deleted; shown in Trash view only.               |
| <b>tags</b>           | [String]      | User-defined labels for filtering.                    |
| <b>lastAccessedAt</b> | Date          | Used for "Recently Viewed" sorting in Dashboard.      |

### 7.1.3 Collection: folders

| Field            | Type          | Description                              |
|------------------|---------------|------------------------------------------|
| <b>name</b>      | String        | Folder display name.                     |
| <b>workspace</b> | String        | Parent workspace label (required).       |
| <b>parentId</b>  | ObjectId null | Supports infinitely nested folder trees. |
| <b>userId</b>    | String        | Owner's Clerk ID.                        |



## 7.2 ER Diagram



## 7.3 API Endpoints — Full Reference

### 7.3.1 Map Routes (/api/maps)

| Method | Endpoint                | Description                                                         |
|--------|-------------------------|---------------------------------------------------------------------|
| GET    | /maps/list              | Fetch all maps owned by authenticated user; supports query filters. |
| POST   | /maps/create            | Create a Map and atomically create its root Node.                   |
| PATCH  | /maps/:mapId/attributes | Update isFavorite, isTrashed, tags, name, etc.                      |
| POST   | /maps/:mapId/duplicate  | Deep-clone map + all nodes with new ObjectIds.                      |
| PATCH  | /maps/bulk-attributes   | Update attributes on multiple maps in one call.                     |

### 7.3.2 Node Routes (/api/nodes)

| Method | Endpoint            | Description                                            |
|--------|---------------------|--------------------------------------------------------|
| GET    | /nodes/map/:mapId   | Fetch all nodes for a map (flat array).                |
| POST   | /nodes/map/:mapId   | Create a single new child node.                        |
| PATCH  | /nodes/:id          | Update name, isExpanded, parentId, notes, status, etc. |
| PATCH  | /nodes/:id/position | Update x, y coordinates after free-drag.               |
| DELETE | /nodes/:id          | Recursively delete node and all descendants.           |
| POST   | /nodes/bulk-update  | Batch update multiple nodes (used by copy/paste).      |

|      |                    |                                                    |
|------|--------------------|----------------------------------------------------|
| POST | /nodes/bulk-create | Insert an array of nodes atomically (Bulk Import). |
|------|--------------------|----------------------------------------------------|

### 7.3.3 Folder Routes (/api/folders)

| Method | Endpoint                   | Description                                           |
|--------|----------------------------|-------------------------------------------------------|
| GET    | /folders/workspaces/list   | Return distinct workspace names for the user.         |
| PATCH  | /folders/workspaces/rename | Rename workspace string on all maps and folders.      |
| DELETE | /folders/workspaces/:name  | Delete workspace and all its content.                 |
| GET    | /folders/list              | List folders by workspace and optional parentId.      |
| POST   | /folders/create            | Create a new folder inside a workspace.               |
| DELETE | /folders/:id               | Recursively delete folder and all child folders/maps. |

## 7.4 D3 Layout Algorithm (LLD Detail)

The layout engine in `flowUtils.js` supports three modes:

### Horizontal Layout (Default — Column-Width Algorithm)

1. `d3.stratify()` converts the flat array into a D3 hierarchy.
2. `tree().nodeSize([1,1]).separation()` assigns abstract Y positions; `gap = (heightA + heightB) / 2 + 40px`.
3. A **Column-Width Map** is built: for each depth level, the maximum node text width is measured using a canvas 2D context (`measureTextPx()`).
4. Each column's X offset = previous column's X + previous column's max width + 50px gap.
5. This prevents label overlap even with very long node names.

### Radial Layout

1. `tree().size([2*Math.PI, RADIUS])` maps the hierarchy onto a circle (angle, radius).
2. Root placed at (0, 0); children spread at equal angular intervals.
3. Polar  $\rightarrow$  Cartesian:  $x = r * \cos(\theta - \pi/2)$ ,  $y = r * \sin(\theta - \pi/2)$ .

**Vertical Layout**

Standard top-down tree. `tree().nodeSize([200, 110])` — sibling gap 200px, depth gap 110px.

## 7.5 Bulk Import Parser Algorithm

Defined in `parseIndentedText()` inside `flowUtils.js`:

Listing 5: Bulk Import Indentation Parser

```

1 // Input: raw text pasted by user
2 // Output: flat array [{id, parentId, name}]
3 const stack = [{ depth: -1, id: targetParentId }]; // sentinel
4 lines.forEach(line => {
5     const depth = countIndent(line); // tabs=4, spaces=1
6     const name = line.trim().replace(/^[ -]*\s+/, ''); //
7     strip bullets
8     // Pop stack until we find the correct parent depth
9     while (stack.length > 1 && stack[stack.length-1].depth >=
10         depth) stack.pop();
11     const parentId = stack[stack.length - 1].id;
12     const id = generateId();
13     newNodes.push({ id, parentId, name, isExpanded: true });
14     stack.push({ depth, id });
15 });

```

Example input → output:

**Input text**

```

Machine Learning
  Supervised
    Regression
    Classification
  Unsupervised

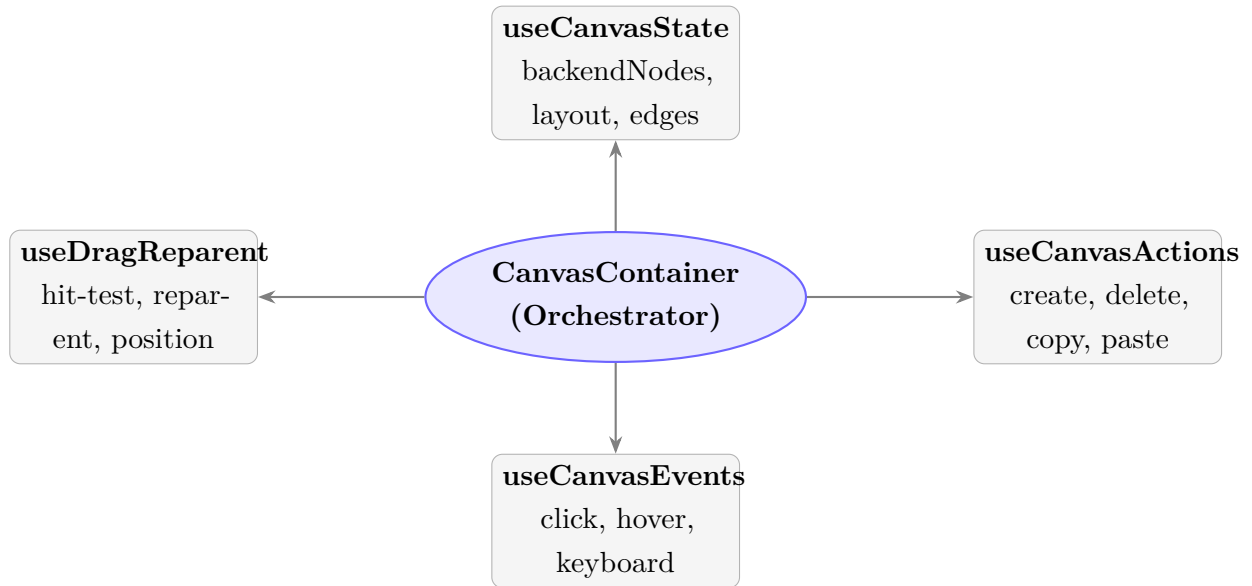
```

Produces: 5 node documents, each with a `parentId` pointing to the correct ancestor.

## 8 Frontend & Interaction Design

### 8.1 Hook Architecture

`CanvasContainer.jsx` delegates all logic to four purpose-built hooks:



## 8.2 useCanvasState: The Sync Engine

This hook is the single source of truth. Whenever `backendNodes`, `layoutMode`, or `colorPalette` changes, a `useEffect` fires and:

1. Calls `filterExpandedNodes()` to prune collapsed subtrees.
2. Calls `buildReactFlowData()` to recompute positions.
3. Enriches each node with runtime callbacks: `onToggle`, `onDraftConfirm`, `onDraftCancel`.
4. Calls `setNodes()` and `setEdges()` to push into React Flow.

## 8.3 D3StyleNode: The Node Visual Component

Each node on canvas is rendered by `D3StyleNode.jsx`. Key design decisions:

- **Dynamic width:** Node width is calculated using a Canvas 2D `measureText()` call — no fixed widths.
- **Inline editing:** Double-click flips the node into an `<input>` field. `Enter` confirms, `Escape` cancels.
- **Glow ring:** On hover or selection, a coloured glow ring (matching branch colour) appears via CSS `box-shadow`.
- **Drop-target pulse:** When a node is dragged over, a pulsing blue ring animates via `pulseRing` keyframe.
- **Status icons:** Lucide React icons rendered based on the `status` field value.

## 8.4 Keyboard Interaction Map

| Key         | Condition                         | Action                                       |
|-------------|-----------------------------------|----------------------------------------------|
| Tab         | Node focused                      | Create child node (open input as draft)      |
| Enter       | Node focused                      | Create sibling node                          |
| F2          | Node focused                      | Rename node (inline edit)                    |
| Delete      | Node focused                      | Delete node + all descendants (with confirm) |
| Ctrl+C      | Node focused                      | Copy node branch to clipboard                |
| Ctrl+V      | Node focused                      | Paste branch as children                     |
| Paste       | Canvas focused, text in clipboard | Trigger Bulk Import parser                   |
| Right-click | Any node                          | Open context menu (FloatingToolbar)          |

## 8.5 Context Menu — FloatingToolbar

Right-clicking any node opens a **FloatingToolbar** positioned at cursor coordinates, offering: Rename, Add Child, Delete, Copy, Paste, Bulk Import, Open Notes, Open Links, Open Files, Open AI.

# 9 Non-Functional Requirements

| ID            | Category    | Requirement                                                              | Target        |
|---------------|-------------|--------------------------------------------------------------------------|---------------|
| <b>NFR-01</b> | Performance | Canvas renders map with 500 nodes without dropping below 30 FPS          | $\geq 30$ FPS |
| <b>NFR-02</b> | Performance | API response for <code>GET /nodes/map/:id</code> for a 500-node map      | $< 300$ ms    |
| <b>NFR-03</b> | Performance | Bulk Import of a 200-line indented text file                             | $< 1$ s       |
| <b>NFR-04</b> | Security    | All protected routes return HTTP 401 without a valid <code>userId</code> | Always        |
| <b>NFR-05</b> | Security    | Users cannot access maps owned by other <code>userIds</code>             | Always        |
| <b>NFR-06</b> | Security    | CORS configured to allow only the production frontend domain             | Always        |
| <b>NFR-07</b> | Reliability | Reparent drag failure triggers automatic rollback from backend re-fetch  | Always        |
| <b>NFR-08</b> | Usability   | Keyboard shortcuts (Tab, Enter, F2, Delete) are functional on all nodes  | Always        |
| <b>NFR-09</b> | Usability   | Inline node editing activates within 50ms of double-click                | $< 50$ ms     |

---

**NFR-10** Scalability      Adding a new layout mode requires only changes to `flowUtils.js`      Isolated

---

## 10 Testing Requirements

### 10.1 Strategy

- **Unit Tests (Jest):** Pure functions — `parseIndentedText()`, `buildReactFlowData()`, `filterExpandedNodes()`, `isDescendantOf()`.
- **Integration Tests (Supertest):** All REST endpoints with mock `userId` injection.
- **E2E Tests (Cypress/Playwright):** Full user journeys on a running dev server.

### 10.2 Key Test Cases

| ID    | Scenario                                               | Expected Result                                                  |
|-------|--------------------------------------------------------|------------------------------------------------------------------|
| TC-01 | <i>Bulk import of 5-level indented text</i>            | 5-level tree created with correct <code>parentId</code> chain    |
| TC-02 | <i>Drag a node onto its own descendant</i>             | Drop target highlighted is skipped; no re-parent                 |
| TC-03 | <i>Drag re-parent succeeds</i>                         | <code>parentId</code> updated in DB; canvas re-renders instantly |
| TC-04 | <i>API call fails during re-parent</i>                 | Frontend reverts to pre-drag state by re-fetching                |
| TC-05 | <i>GET /nodes/map/:id by non-owner</i>                 | HTTP 401 returned                                                |
| TC-06 | <i>Create map</i>                                      | Map + root node both created; <code>rootNodeId</code> is set     |
| TC-07 | <i>Delete node with 3 descendants</i>                  | All 4 nodes removed from DB; canvas updates                      |
| TC-08 | <i>Collapse a node (<code>isExpanded=false</code>)</i> | Children hidden from canvas; expand restores them                |
| TC-09 | <i>Switch layout from Horizontal to Radial</i>         | Canvas re-renders in radial polar layout                         |
| TC-10 | <i>Node with notes shows [Icon] badge</i>              | Badge visible in <code>D3StyleNode</code> display text           |

---

## 11 Deployment & Planning

## 11.1 Deployment Architecture

| Component             | Platform              | Notes                                         |
|-----------------------|-----------------------|-----------------------------------------------|
| Frontend (React/Vite) | Vercel                | Auto-deploy from Git; CDN-served globally     |
| Backend (Express)     | Render / Railway      | Node.js v18 Docker container, port 5000       |
| Database              | MongoDB Atlas         | Shared M0 (free) or Serverless tier           |
| Auth                  | Clerk                 | Free tier, JWT-based                          |
| File Storage          | TBD (Cloudinary / S3) | For <code>files[]</code> attachments per node |

## 11.2 Development Milestones

| Phase         | Duration | Deliverable                      | Status      |
|---------------|----------|----------------------------------|-------------|
| Architecture  | 1 week   | Repo, schema, route skeleton     | Done        |
| Backend Core  | 1 week   | All REST endpoints + services    | Done        |
| Canvas Engine | 2 weeks  | React Flow + D3 layout + 3 modes | Done        |
| Bulk Import   | 1 week   | Parser + bulk-create endpoint    | Done        |
| Dashboard UI  | 1 week   | Dashboard, MyMaps, Workspace     | Done        |
| Rich Nodes    | 1 week   | Notes, Links, Files, Status      | In Progress |
| AI Features   | 2 weeks  | Node generation via LLM API      | Planned     |
| E2E Testing   | 1 week   | Cypress test suite               | Planned     |
| Production    | 1 week   | Vercel + Render deploy           | Planned     |

## 11.3 Future Enhancements

1. **AI Node Generation:** Select a node and invoke an LLM API to auto-generate a list of relevant child topics.
2. **Real-time Collaboration:** WebSocket layer (Socket.io) so multiple users can edit the same map simultaneously with live cursors.
3. **Export:** Download map as a high-resolution PNG, PDF, or Markdown outline.
4. **Templates:** Pre-built map structures (SWOT Analysis, Project Plan, Study Schedule) loaded via `bulk-create`.
5. **Deadline/Reminders:** Add a `deadline` Date field to nodes and surface overdue items in the Dashboard.

## 12 Conclusion

Axon Flow is architected around one central insight: storing mind-map nodes as a *flat, independently updatable list* in MongoDB — rather than as deeply nested documents —

provides superior performance for both reads and writes, without sacrificing the rich, hierarchical user experience.

The frontend resolves this flat representation into a beautiful visual tree at runtime using D3-Hierarchy's battle-tested layout algorithms, React Flow's performant Virtual DOM graph, and four purpose-built hooks that cleanly separate state, events, actions, and drag logic.

The result is a system that feels instant to the user, is easy to extend for developers, and is ready to grow into an AI-assisted, collaborative knowledge-mapping platform.

## 12 Appendix — Technology Stack

| Technology                        | Version | Role                                   |
|-----------------------------------|---------|----------------------------------------|
| <b>React</b>                      | 18      | UI framework and Virtual DOM rendering |
| <b>@xyflow/react (React Flow)</b> | 12+     | Interactive node-graph canvas          |
| <b>d3-hierarchy</b>               | 3       | Tree layout (stratify, tree)           |
| <b>Vite</b>                       | 5       | Frontend build tool (HMR, fast build)  |
| <b>Node.js</b>                    | 18      | Backend JavaScript runtime             |
| <b>Express.js</b>                 | 4       | REST API framework                     |
| <b>MongoDB Atlas</b>              | 6       | Cloud NoSQL database                   |
| <b>Mongoose</b>                   | 7       | ODM for MongoDB schema and queries     |
| <b>Clerk</b>                      | latest  | Authentication, JWT, OAuth             |
| <b>Lucide React</b>               | latest  | Status icons in node component         |
| <b>TailwindCSS</b>                | 3       | Dashboard and page-level styling       |



### 13 Project Outputs & Screenshots



Figure 1: AxonFlow Interface - Workspace & Dashboard

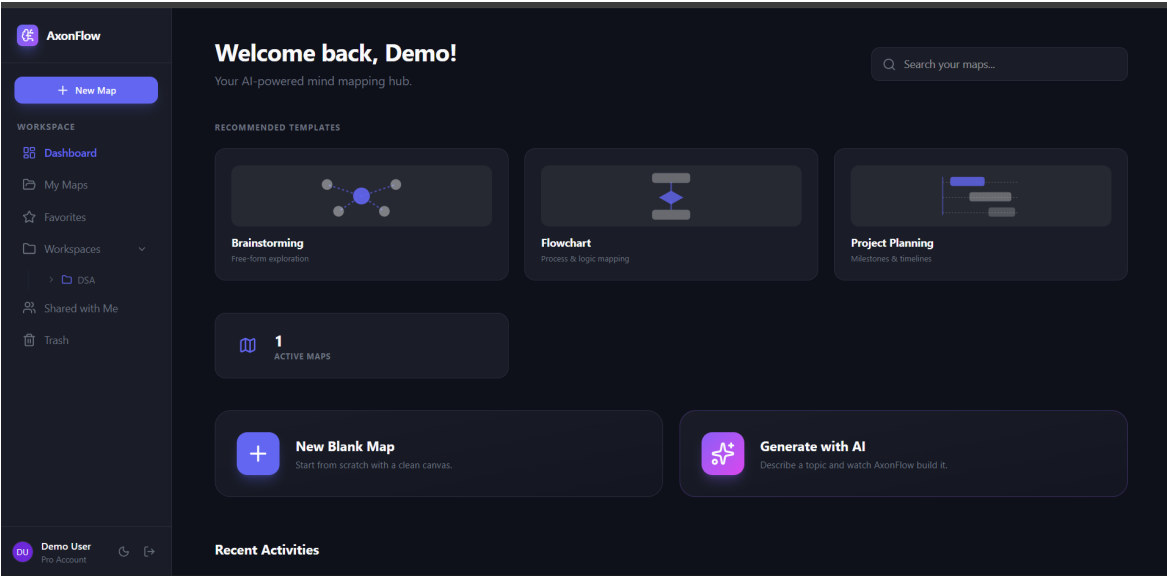


Figure 2: AxonFlow Interface - Mind Map Canvas Editor

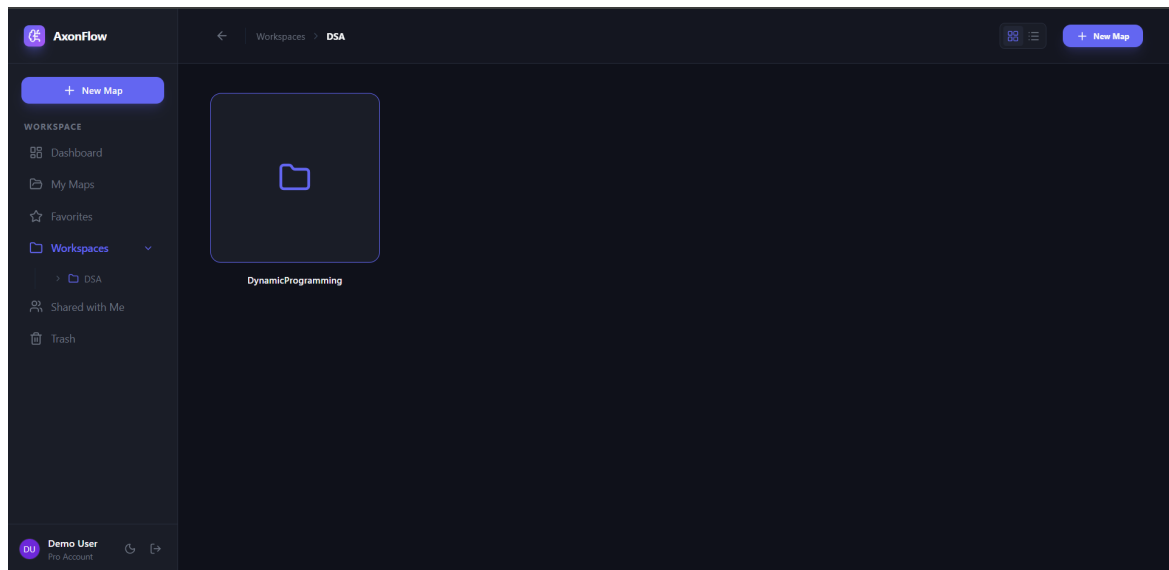


Figure 3: AxonFlow Interface - Node Details and Panels